EXP-6 Retrieval of Similar Items using Vector Embeddings (FAISS)

AIM:

To implement a system for retrieving similar movies using high dimensional vector embeddings with FAISS.

DATASET:

MovieLens Dataset (Movie titles)

PROCEDURE:

Step 1: Install and Import Libraries

```
pip install faiss-cpu sentence-transformers pandas

import faiss
import pandas as pd
from sentence_transformers import SentenceTransformer
```

Step 2: Load Dataset

```
movies = [
    "The Matrix",
    "Inception",
    "Interstellar",
    "The Dark Knight",
    "Titanic",
    "Avatar",
    "The Avengers"
]

df = pd.DataFrame(movies, columns=["title"])
print(df.head())
```

Output:

```
            title
0        The Matrix
1         Inception
2      Interstellar
3   The Dark Knight
4           Titanic
```

Step 3: Load Embedding Model

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

Step 4: Convert Text to Embeddings

```
embeddings = model.encode(df['title'].tolist())
print(embeddings.shape)
```

Output:

```
(7, 384)
```

Step 5: Create FAISS Index

```
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
```

Step 6: Perform Similarity Search

```
query = "science fiction movie"
query_vec = model.encode([query])

distances, indices = index.search(query_vec, k=3)

results = df.iloc[indices[0]]
print(results)
```

Output:

	title
0	The Matrix
2	Interstellar
1	Inception

Step 7: Observations

- FAISS retrieves semantically similar movies
- Embeddings capture meaning, not just keywords
- Results are relevant to query context

Output:
Top 3 similar movies retrieved successfully.

RESULT:

FAISS retrieves similar movies efficiently using high-dimensional embeddings. It enables accurate semantic search for recommendation systems.